

Analyzing Growth Functions

A Visual, Mathematical Comparison of $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, and $O(2^n)$

Why Compare Growth Functions?

1

Predict Performance

Growth functions describe how an algorithm's running time or space scales as input size n increases.

2

Compare Algorithms Fairly

Asymptotic notation strips away hardware and constant-factor noise, letting us compare designs on structure alone.

3

Guide Algorithm Selection

Knowing which growth class an algorithm belongs to tells us whether it will remain practical as data scales up.

The Five Growth Functions



$O(\log n)$

Logarithmic

e.g. Binary search



$O(n)$

Linear

e.g. Linear scan



$O(n \log n)$

Linearithmic

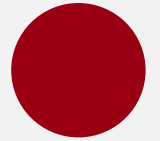
e.g. Merge sort



$O(n^2)$

Quadratic

e.g. Bubble sort

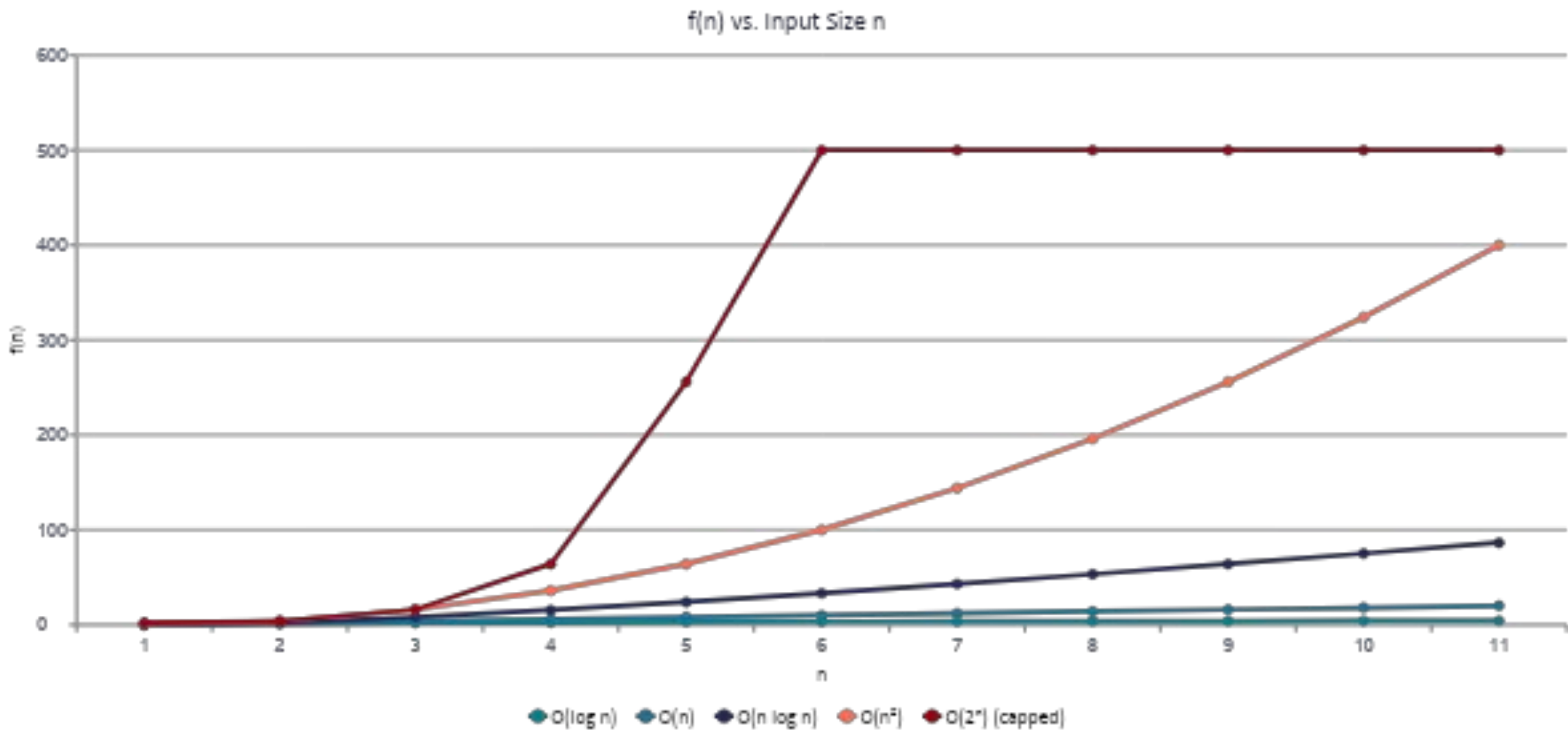


$O(2^n)$

Exponential

e.g. Subset enumeration

Growth Behavior as Input Size Increases



Function Values at Increasing n

n	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(2^n)$
10	3.3	10	33	100	1,024
20	4.3	20	86	400	1,048,576
30	4.9	30	147	900	1.1e+9
40	5.3	40	213	1600	1.1e+12
50	5.6	50	282	2500	1.1e+15

Notice how $O(2^n)$ rockets past all other functions by $n = 30$ — at $n = 50$ it already exceeds a quadrillion operations.

Which Functions Stay Practical at Scale?

Practical for Large n

$O(\log n)$ — grows almost flat; doubling n adds one step

$O(n)$ — scales directly with input, easy to reason about

$O(n \log n)$ — the practical ceiling for sorting/searching at scale

Impractical for Large n

$O(n^2)$ — usable for small/medium n , but doubling n quadruples work

$O(2^n)$ — explodes almost immediately; only viable for tiny n

Rule of thumb — if n can be large, exponential and unoptimized quadratic algorithms should be avoided or redesigned

Linking Growth Behavior to Algorithm Selection

Small, fixed input

Any growth class works — optimize for simplicity and readability.

Large or unbounded input

Favor $O(\log n)$, $O(n)$, or $O(n \log n)$ algorithms to keep runtime manageable.

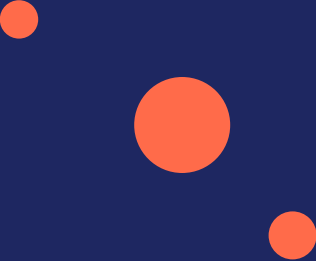
Search / lookup tasks

Use divide-and-conquer or indexed structures ($O(\log n)$) over linear scans when possible.

Sorting large datasets

$O(n \log n)$ algorithms like merge sort or heap sort outperform $O(n^2)$ options like bubble sort.

Conclusion



Asymptotic growth functions reveal how algorithms truly scale, independent of hardware.

$O(\log n)$, $O(n)$, and $O(n \log n)$ remain efficient and practical even as input size grows very large.

$O(n^2)$ and $O(2^n)$ become bottlenecks quickly and should be avoided or replaced for large-scale problems.

Understanding growth behavior is therefore essential to selecting the right algorithm for the right problem.